

On the Effectiveness of LayerNorm Tuning for Continual Learning in Vision Transformers

Thomas De Min¹ Massimiliano Mancini¹ Karteek Alahari² Xavier Alameda-Pineda² Elisa Ricci^{1,3}

¹University of Trento ²Inria[†] ³Fondazione Bruno Kessler

Abstract

State-of-the-art rehearsal-free continual learning methods exploit the peculiarities of Vision Transformers to learn task-specific prompts, drastically reducing catastrophic forgetting. However, there is a tradeoff between the number of learned parameters and the performance, making such models computationally expensive. In this work, we aim to reduce this cost while maintaining competitive performance. We achieve this by revisiting and extending a simple transfer learning idea: learning task-specific normalization layers. Specifically, we tune the scale and bias parameters of LayerNorm for each continual learning task, selecting them at inference time based on the similarity between task-specific keys and the output of the pre-trained model. To make the classifier robust to incorrect selection of parameters during inference, we introduce a two-stage training procedure, where we first optimize the task-specific parameters and then train the classifier with the same selection procedure of the inference time. Experiments on ImageNet-R and CIFAR-100 show that our method achieves results that are either superior or on par with the state of the art while being computationally cheaper.¹

1. Introduction

The goal of class-incremental learning (IL) [42] is to learn a classifier over a sequence of learning steps (or tasks), where each one contains a different set of classes. One of the main challenges of IL is to avoid forgetting old knowledge when learning new categories, a phenomenon called catastrophic forgetting [24, 37]. One of the most effective ways to mitigate forgetting is to store a subset of old class data and use them to perform rehearsal of old knowledge [19, 32, 33, 39, 42]. However, storing samples of old classes is not always possible due to potential memory constraints [18] or privacy issues [46, 51].

[†]Univ. Grenoble Alpes, CNRS, Grenoble INP, LJK, 38000 Grenoble, France.

¹Code is available at <https://github.com/tdeamin16/Continual-LayerNorm-Tuning>.

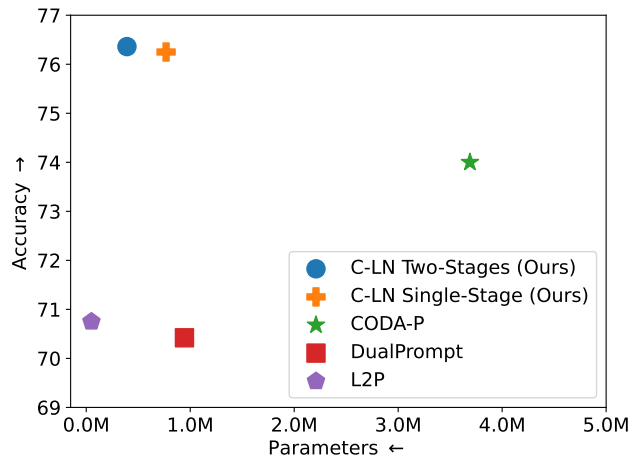


Figure 1: Continual-LayerNorm’s efficiency compared to state-of-the-art methods. Our approach, C-LN, improves the accuracy in ImageNet-R over a 10-task benchmark while training fewer parameters.

Recent works [43, 45, 46] demonstrated how exploiting pre-trained Vision Transformers (ViT) [16] can lead to competitive IL results even in a rehearsal-free setting. The key idea behind these models is to freeze the pre-trained backbone and learn specific input prompts for each learning step. During inference, the model selects which prompts to use based on step-specific keys and the activations of the pre-trained backbone for the current input. Notably, the visual prompts of one learning step do not interfere with the old ones [45, 46], drastically reducing catastrophic forgetting. Despite their advantages, these methods are inefficient in terms of computational cost: the number of learnable parameters (e.g., prompts) impacts the results, and a large number of prompts is required to achieve state-of-the-art performance [43].

In this work, we study whether it is possible to sidestep this computational burden while maintaining competitive performance. To achieve this, we propose to revisit and extend a simple transfer learning idea, i.e., learning spe-

cific normalization layers. Inspired by recent works on efficient ViT fine-tuning [4], we learn specific scale and bias parameters of LayerNorm [2] for each incremental learning step. As in prompt learning approaches [46], we learn task-specific keys, training them to mimic the output of the frozen pre-trained ViT for samples of their specific learning step. During inference, we then use the parameters whose associated key is the closest to the pre-trained model output for the given input sample. Finally, we make the classifier more robust to selection errors via a two-stage training pipeline, where we first use the ground-truth task identity to tune the LayerNorm parameters and then simulate the inference-time selection procedure to train the classifier. We name our approach **Continual LayerNorm Tuning**, **C-LayerNorm**.

Experiments on CIFAR-100 [27] and ImageNet-R [20], show that C-LayerNorm achieves either the same or superior results of prompt tuning strategies, while providing up to 90% reduction over the number of trained parameters compared to the state of the art, CODA-Prompt [43]. This is reported in Fig. 1 showing the accuracy vs the number of parameters trained for different approaches on the ImageNet-R dataset, with our C-LayerNorm achieving the best tradeoff.

In summary, we make the following contributions:

- We propose C-LayerNorm, a rehearsal-free continual learning method that tunes task-specific LayerNorm parameters for each incremental learning step, selecting them during inference.
- We design a variant of the approach for single-stage inference, where the selection is performed within each LayerNorm. Despite tuning more parameters, this strategy leads to an inference speed two times faster than the current state of the art.
- Experiments on standard benchmarks show that both single- and two-stage approaches consistently achieve state-of-the-art performance, with the best tradeoff between performance and the number of parameters.

2. Related Work

Continual Learning. Early approaches for continual learning tackled catastrophic forgetting through regularization objectives, e.g., preventing changes of model parameters important for old tasks [1, 13, 26, 49] or via distilling the old model’s activations [9–11, 17, 31, 42]. These methods can be coupled with a memory buffer, storing samples of old tasks/classes for performing knowledge rehearsal [12, 32, 33, 39, 50]. While this strategy is extremely effective for incremental learning, both in terms of accuracy and efficiency [19, 38], it is not viable in case of privacy constraints, preventing storing data of past tasks. To

deal with such cases, the community focused on rehearsal-free continual learning, with different works proposing various strategies, ranging from strong regularization objectives on the model’s activations [18], to self-supervised objectives [51], and modeling the semantic drift of the network’s representation [48]. More recently, ViT-specific approaches showed that rehearsal is not needed to achieve state-of-the-art performance in continual learning. These models tune a subset of the network parameters for each task/incremental step, selecting the most relevant ones during inference. The seminal work in this direction is Learning to Prompt [46] which proposes to perform visual prompt tuning [23] and prepend a set of learnable tokens as input to the Vision Transformer. Follow-up works improve either the way prompts are injected within the model [45] or trained across incremental learning steps [43]. These approaches achieve impressive results but inherit a trade-off between several task-specific parameters and performance.

Differently from these works, in this paper, we propose to learn specific normalization layers, showing that this strategy achieves either comparable or superior performance while requiring much fewer parameters than the state-of-the-art prompting methods [43, 45].

Parameter-Efficient Fine-Tuning. Due to its relevance for resource-constrained settings, parameter-efficient fine-tuning, i.e., tuning a pre-trained model on a specific task without either re-training it from scratch or adding a large number of parameters, is a widely studied topic in machine learning. In computer vision, early works explored the use of adapters to modify the activations of residual blocks in convolutional neural networks [40, 41]. Subsequent efforts further reduced the number of parameters, either by extracting task-specific subnetworks [5, 34–36] or by factorizing the network’s layers [7]. For vision transformers, multiple techniques have been borrowed from the natural language processing literature, such as standard [21] and low-rank adapters [22], or prompt-tuning via additional input tokens [23, 29] or modifying the images themselves [3].

In this work, we follow a different path, learning specific normalization parameters per incremental step. This shares similarities with previous works tuning specific batch-normalization parameters [6], layer normalization ones [4], or by introducing covariance normalization layers [30]. However, here we focus on a different problem formulation, demonstrating that normalization strategies are an effective way to perform rehearsal-free IL in vision transformers.

3. Method

In this section, we first describe the problem formulation (Sec. 3.1), and how LayerNorm can be used for continual learning (Sec. 3.2). We then present our two-stage (Sec. 3.3) and single-stage (Sec. 3.4) approaches. Finally, we describe

our full training procedure and how we make the classifier more robust to selection errors during inference (Sec. 3.5).

3.1. Notation and problem formulation

The goal of continual learning is to learn a model from a sequence of non-stationary datasets, performing training over multiple steps. Formally, we receive sequential data $\mathcal{D} = \{D_1, \dots, D_T\}$, where T is the number of incremental learning steps/tasks and D_t is the dataset available at timestep t . Each dataset is a collection of tuples such that $D_t = \{(\mathbf{x}_i^t, y_i^t)\}_{i=1}^{n_t}$. $\mathbf{x}_i^t \in \mathcal{X}$ is the i -th image of task t in the image space $\mathcal{X}$, and $y_i^t \in \mathcal{Y}_t$ is its corresponding label in $\mathcal{Y}_t$ in the case of the image classification problem. Note that the semantic space of the dataset D_t , i.e., $\mathcal{Y}_t$, does not overlap with the one at a different learning step, i.e., $\mathcal{Y}_t \cap \mathcal{Y}_u = \emptyset$ for each step $t, u \in \{1, \dots, T\}$ with $t \neq u$.

Given $\mathcal{D}$, we aim to learn a function $f_\theta^T : \mathcal{X} \rightarrow \mathcal{Y}$, parametrized by θ that maps images in the space $\mathcal{X}$ to labels in the set $\mathcal{Y}$, where $\mathcal{Y} = \bigcup_{t=1}^T \mathcal{Y}_t$. The literature distinguishes between task- and class-incremental learning depending on the availability of task identities during inference. In this work, we focus on the latter, where task identities are unknown and f_θ^T performs predictions over the full set $\mathcal{Y}$. In the following, we will use the terms *incremental step* and *task* interchangeably.

To tackle this problem, we split the parameters θ into two sets: θ_g containing global (frozen parameters) and a set of task-specific parameters θ_s , fine-tuned for each learning step, i.e., $\theta_s = \{\theta_s^1, \dots, \theta_s^T\}$. While standard approaches use prompts as θ_s , their performance is influenced by the number of task-specific parameters they train [43]. In this work, we take a different direction, and show that tuning LayerNorm can reduce the number of learnable parameters without sacrificing the final accuracy.

3.2. Continual Learning with LayerNorm

Inspired by the multi-domain learning [6] and the transfer learning [4] literatures, we address catastrophic forgetting by learning specific normalization layer weights for each incremental learning step. In the case of ViTs, the normalization layers are LayerNorm ones [2], in which we tune the scale and bias parameters. Formally, given as input a feature vector z , LayerNorm [2] is defined as:

$$\text{LayerNorm}(z) = \frac{z - \mathbb{E}[z]}{\sqrt{\text{Var}[z] + \epsilon}} \odot \gamma^\ell + \beta^\ell, \quad (1)$$

where γ^ℓ and β^ℓ are the learnable scale and bias parameters for layer ℓ , and ϵ is a small value to prevent numerical problems. For the purpose of continual learning, we aim to learn a specific LayerNorm for each incremental learning step. We thus revise Eq. (1) as

$$\text{C-LayerNorm}(z, t) = \frac{z - \mathbb{E}[z]}{\sqrt{\text{Var}[z] + \epsilon}} \odot \gamma_t^\ell + \beta_t^\ell, \quad (2)$$

where t is the incremental step indicator and γ_t^ℓ and β_t^ℓ are its specific scale and bias parameters for layer ℓ . Since we allocate a set of LayerNorm parameters for each task, during inference we need to infer task identities to select the best weights for the given input sample. In the following, we describe two ways of selecting parameters during inference: two-stage (Fig. 2a), where we first infer the identity and then perform the prediction, and single-stage (Fig. 2b), where we estimate the identity within each C-LayerNorm.

3.3. Two-stage C-LayerNorm

In this case, we follow prompt-learning approaches for continual learning [46] and instantiate a learnable key vector for each incremental step (Fig. 3). The key vector will then be used during inference to estimate the task identity. Formally, for each dataset D_t , we instantiate a key parameter $\theta_{\text{SEL}}^t \in \mathbb{R}^d$ where d is the output dimension of the cls token of the frozen pre-trained model. We learn θ_{SEL}^t by minimizing the following objective:

$$\mathcal{L}_{\text{key}}^{2\text{-stage}}(\theta_{\text{SEL}}^t) = \frac{1}{|D_t|} \sum_{(x, y) \in D_t} 1 - \frac{f_\theta^0(x)^\top \theta_{\text{SEL}}^t}{\|f_\theta^0(x)\| \cdot \|\theta_{\text{SEL}}^t\|}, \quad (3)$$

where, for simplicity, f_θ^0 denotes the pre-trained model, mapping images to their corresponding cls embeddings. Eq. (3) maximizes the average cosine similarity between the task-specific key θ_{SEL}^t and the embedding of the dataset samples, as extracted from the pre-trained model.

During inference, we estimate the task identity $\hat{t}$ (and thus the specific set of C-LayerNorm parameters to use) by computing the cosine similarity between the keys and the embedding of the input sample $\mathbf{x}$, i.e.,

$$\hat{t} = \arg \max_{t \in \{1, \dots, T\}} \frac{f_\theta^0(\mathbf{x})^\top \theta_{\text{SEL}}^t}{\|f_\theta^0(\mathbf{x})\| \cdot \|\theta_{\text{SEL}}^t\|}. \quad (4)$$

Note that, given a sample, all C-LayerNorm of the model uses the same unique task identity, thus we first estimate the task ID with a forward pass over f_θ^0 and then perform a second forward on f_θ^T to get the semantic prediction (Fig. 2a).

3.4. Single-stage C-LayerNorm

An alternative to the two-stage procedure is to compute the task ID on the fly, on each C-LayerNorm, avoiding multiple sequential forward passes (Fig. 2b). To achieve this, we instantiate one task-specific key for each C-LayerNorm, performing a similar procedure as in Section 3.3, but independently at each layer. Formally, let us denote as z_ℓ the activation in input to the C-LayerNorm ℓ , and as $\theta_{\text{SEL}}^{t, \ell}$ the selection key at layer ℓ for step t . Similarly to the previous case, we perform the parameters' selection at layer ℓ as

$$\hat{t}_\ell = \arg \max_{t \in \{1, \dots, T\}} \frac{(\alpha_\ell^t \odot z_\ell^0)^\top \theta_{\text{SEL}}^{t, \ell}}{\|\alpha_\ell^t \odot z_\ell^0\| \cdot \|\theta_{\text{SEL}}^{t, \ell}\|}. \quad (5)$$

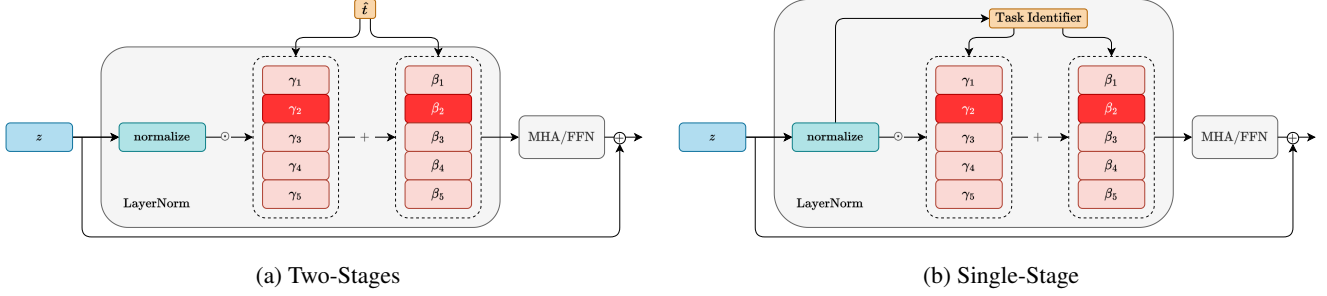


Figure 2: Two variants of our approach. On the left, every LayerNorm in the Two-Stages version must be provided with the task identity to select the set of parameters. On the right, instead, our Single-Stage LayerNorm estimates task identities on the fly. By doing so, we do not need to compute task identities a priori, thus almost halving the computational burden.

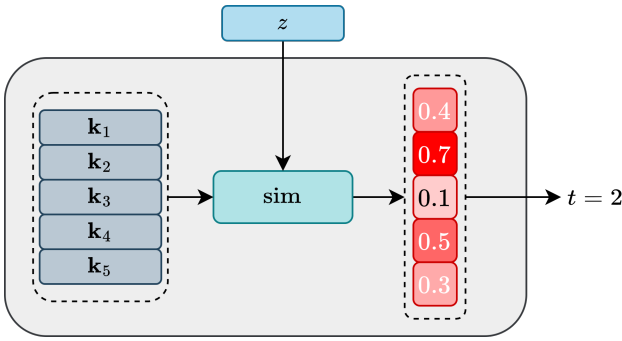


Figure 3: Task identifier structure. We compute the similarity between the feature vector z and all the learned task vectors. Each comparison leads to a similarity score. We infer task identities as the index of the vector that has the highest similarity score with the input features.

where z_ℓ^0 is the cls token embeddings at the same layer as given by the pre-trained model f_θ^0 , and $\alpha_\ell^t \in \mathbb{R}^d$ is a learnable attention vector scaling the values of the input. Similarly to [43], we found it beneficial to introduce this attention vector, adding flexibility to the selection procedure.

We train both the keys and the attentions using an input dataset D_t , maximizing the cosine similarity between the scaled cls token embeddings and the keys:

$$\mathcal{L}_{\text{key},\ell}^{\text{1-stage}}(\theta_{\text{SEL}}^{t,\ell}, \alpha_\ell^t) = \frac{1}{|D_t|} \sum_{(x,y) \in D_t} 1 - \frac{(\alpha_\ell^t \odot z_\ell^0)^\top \theta_{\text{SEL}}^{t,\ell}}{\|\alpha_\ell^t \odot z_\ell^0\| \cdot \|\theta_{\text{SEL}}^{t,\ell}\|} \quad (6)$$

and thus, considering L C-LayerNorm layers:

$$\mathcal{L}_{\text{key}}^{\text{1-stage}}(\theta_{\text{SEL}}^t) = \frac{1}{L} \sum_{\ell=1}^L \mathcal{L}_{\text{key},\ell}^{\text{1-stage}}(\theta_{\text{SEL}}^{t,\ell}, \alpha_\ell^t), \quad (7)$$

where here, for simplicity, we denote as θ_{SEL}^t the full set of task-specific selection parameters, i.e., $\theta_{\text{SEL}}^t = \{\theta_{\text{SEL}}^{t,\ell}, \alpha_\ell^t\}_{\ell=1}^L$. While this formulation requires additional

parameters, it has two main advantages: i) it almost halves the inference time, as there is no need to perform sequential forward passes; ii) it allows training all parameters end-to-end, as the gradient flows to the selection keys and attention vectors through the intermediate representations.

3.5. Training pipeline

Summarizing the steps before, given a dataset D_t at timestep t , we define its specific set of parameters as $\theta_s^t = \{\theta_{\text{LN}}^t, \theta_{\text{SEL}}^t, \theta_{\text{CLS}}^t\}$, where θ_{LN}^t are the parameters of C-LayerNorm for all L layers of the network, $\theta_{\text{LN}}^t = \{\gamma_\ell^t, \beta_\ell^t\}_{\ell=1}^L$, θ_{SEL}^t are the parameters involved in the selection process (i.e. keys for the two-stage variant, layer-wise keys, and attention vectors for the single stage one). Moreover, at each time step, we introduce θ_{CLS}^t , the parameters of the classifier necessary to recognize the new classes.

Practically, we may define a single objective, learning θ_{CLS}^t and θ_{LN}^t together. However, we found that the mismatch between the set of parameters used during training (i.e., ground-truth task) and inference (i.e., estimated), results in generalization problems, harming the performance of the model. This is shown in Figure 4, where there is a clear gap between the performance of the oracle task selector and the estimated identities via task-specific keys. To reduce this issue we propose a two-stage training procedure, where we first tune the task-specific parameters and then fine-tune the classifier using the same parameters selection procedure adopted during inference.

C-LayerNorm parameters training. To tune the task-specific parameters, we use D_t and minimize

$$\mathcal{L}(\theta_s^t) = \mathcal{L}_{\text{CE}}(\theta_{\text{LN}}^t, \theta_{\text{CLS}}^t) + \mathcal{L}_{\text{key}}(\theta_{\text{SEL}}^t), \quad (8)$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(\theta_{\text{LN}}^t, \theta_{\text{CLS}}^t) = -\frac{1}{|D_t|} \sum_{(x,y) \in D_t} \log f_\theta^t(x)[y]. \quad (9)$$

Above, $f_{\theta}^t(x)[y]$ is the prediction of the model for label y . At each incremental step, we minimize Eq. (8) only w.r.t. the task-specific parameters in θ , i.e., θ_s^t . Following [43], also for the classifier we update only the parameters of new classes θ_{CLS}^t , excluding the others from the backpropagation step. Note that, during the forward pass of C-LayerNorm, we use the ground-truth task identity to ensure that these parameters are optimal and truly specific for the current task.

Classifier refinement. After estimating the task-specific parameters for task selection and C-LayerNorm, we re-initialize the classifier and fine-tune it to be more robust to selection errors during inference. Specifically, we use the cross-entropy loss as defined in Eq. (9), training only θ_{CLS}^t with all the other parameters frozen (i.e., θ_{LN}^t and θ_{SEL}^t) while performing task-selection as during inference time. Note that, by not using the ground-truth identifier, we are exposing the classifier to the potential selection mistakes that the model will experience during inference. This makes the model more robust to selection errors, providing improvements in the overall performance, as shown in Figure 4 and in Sec. 4.

4. Experiments

Here, we first describe our experimental setting (Sec. 4.1) and compare C-LayerNorm with the state-of-the-art in IL (Sec. 4.2). Finally, we analyze how the classifier refinement influences the performance, and, qualitatively, the learned task-specific parameters (Sec. 4.3).

4.1. Experimental setting

Datasets. We conduct experiments on two benchmark datasets used by previous works [43, 45, 46]. The first is **CIFAR-100** [27], a 60k samples dataset that spans 100 classes with 32×32 images. As the images have a smaller dimension compared to the input size of ViT, we upscale them to the expected input size. The second dataset we use is **ImageNet-R** [20]. This dataset is challenging, as it contains 30k images spanning 200 classes and 15 different visual domains. This latter characteristic allows for testing the model in a scenario where the pre-training dataset distribution (i.e., ImageNet-1k [15]) differs from the target one.

Metrics. We use the same evaluation metrics of previous works in this field, namely average accuracy and forgetting. The first is defined as the average accuracy on the last trained model, evaluated on the union of all classes learned across all incremental steps. On the other hand, forgetting measures the average amount of information loss. We refer to Wang *et al.* [45] for further details.

Baselines. We consider various baselines for our experiments. The main competitors are state-of-the-art approaches for rehearsal-free continual learning, namely Learning to Prompt (L2P) [46], DualPrompt [45], and CODA-Prompt [43]. L2P allocates distinct prompts to different tasks, inferring task identities using selection parameters. DualPrompt [45] expands L2P by replacing prompt-tuning [28] with prefix-tuning [29]. CODA-Prompt [43] trains the selection parameters end-to-end, with dynamic prompt allocation. For fairness and compatibility of the results², we re-ran these methods using their original code.

To provide a reference upper bound, we report the results of training a model on the whole dataset at once, either fine-tuning all the parameters in the ViT (UB), or just LayerNorm ones (LN-Tune [4]). Similarly, as lower-bound, we consider fine-tuning the ViT on each incremental learning step, without strategies to prevent forgetting (FT). Following previous works, we also report the results of five rehearsal-based approaches, ER [14], BiC [47], GDumb [39], DER [8], and Co²L [12], evaluating them with a replay buffer of 1000 and 5000 images.

Implementation details. We follow previous work [43, 46], and perform our experiments using a ViT-Base backbone [16] pre-trained on ImageNet-1k [15]. To train our method, we use the Adam optimizer [25] with standard beta values, $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The batch size is set to 128 images for all experiments, and all the images are resized to 224×224, training the models for 5 epochs on each step. The learning rate is set to $1e^{-2}$ for ImageNet-R [20] and $5e^{-3}$ for CIFAR-100 [27]. For both single- and two-stage variants, the classifier is trained with a constant learning rate of $1e^{-3}$ for both datasets. Note that, compared to prompt-tuning methods [43, 45, 46], our approach does not introduce any additional hyperparameter.

4.2. Comparison with the state of the art

10-task benchmarks. In Table 1, we report the results of our approach, C-LayerNorm (C-LN), and all baselines for CIFAR-100 and ImageNet-R on a 10-task benchmark (i.e., splitting the classes into 10 groups and learning one group after the other). From top to bottom, we show the upper bound, rehearsal-based methods, regularization-based ones, prompt-based models, and C-LayerNorm (C-LN). For each method and setting, we report accuracy and forgetting³.

Overall, C-LN achieves the best accuracy (or comparable) across all settings, for both the two-stage and single-stage variants. There is a large gap between the performance of rehearsal-based approaches and our model,

²There is a mismatch in the computation of the forgetting metric between [43] and [45]. As in [45], we follow the original definition in [13].

³GDumb [39], has no forgetting value as it trains the network from scratch after each task.

Table 1: Results on CIFAR-100 and ImageNet-R 10-task benchmarks in class-incremental learning. Both variants of our method perform on par with the current state-of-the-art in the first dataset, while setting a new SOTA in ImageNet-R.

Method	Buffer size	CIFAR-100		Buffer size	ImageNet-R	
		Accuracy $\uparrow$	Forgetting $\downarrow$		Accuracy $\uparrow$	Forgetting $\downarrow$
UB	-	90.85 $\pm$ 0.12	-	-	79.13 $\pm$ 0.18	-
LN-Tune	-	92.05	-	-	81.18	-
ER	1000	67.87 $\pm$ 0.57	33.33 $\pm$ 1.28	1000	53.13 $\pm$ 1.29	35.38 $\pm$ 0.52
BiC		66.11 $\pm$ 1.76	35.24 $\pm$ 1.64		52.14 $\pm$ 1.08	36.70 $\pm$ 1.05
GDumb		67.14 $\pm$ 0.37	-		38.32 $\pm$ 0.55	-
DER		61.06 $\pm$ 0.87	39.87 $\pm$ 0.99		55.47 $\pm$ 1.31	34.64 $\pm$ 1.50
Co ² L		72.15 $\pm$ 1.32	28.55 $\pm$ 1.56		53.45 $\pm$ 1.55	37.30 $\pm$ 1.81
ER	5000	82.53 $\pm$ 0.17	16.46 $\pm$ 0.25	5000	65.18 $\pm$ 0.40	23.31 $\pm$ 0.89
BiC		81.42 $\pm$ 0.85	17.31 $\pm$ 1.02		64.63 $\pm$ 1.27	22.25 $\pm$ 1.73
GDumb		81.67 $\pm$ 0.02	-		65.90 $\pm$ 0.28	-
DER		83.94 $\pm$ 0.34	14.55 $\pm$ 0.73		66.73 $\pm$ 0.87	20.67 $\pm$ 1.24
Co ² L		82.49 $\pm$ 0.89	17.48 $\pm$ 1.80		65.90 $\pm$ 0.14	23.36 $\pm$ 0.71
FT	0	33.61 $\pm$ 0.85	86.87 $\pm$ 0.20	0	28.87 $\pm$ 1.36	63.80 $\pm$ 1.50
EWC		47.01 $\pm$ 0.29	33.27 $\pm$ 1.17		35.00 $\pm$ 0.43	56.16 $\pm$ 0.88
LwF		60.69 $\pm$ 0.63	27.77 $\pm$ 2.17		38.54 $\pm$ 1.23	52.37 $\pm$ 0.64
L2P		83.60 $\pm$ 0.59	6.06 $\pm$ 0.18		70.75 $\pm$ 0.23	4.53 $\pm$ 0.64
DualPrompt		81.34 $\pm$ 0.24	7.81 $\pm$ 1.38		70.42 $\pm$ 0.21	5.86 $\pm$ 0.74
CODA-P		86.84 $\pm$ 0.41	5.76 $\pm$ 0.64		74.00 $\pm$ 0.42	6.62 $\pm$ 0.57
C-LN Two-Stages		86.95 $\pm$ 0.37	6.98 $\pm$ 0.43		76.36 $\pm$ 0.51	8.31 $\pm$ 1.28
C-LN Single-Stage	0	86.06 $\pm$ 0.83	8.60 $\pm$ 1.68	0	76.25 $\pm$ 0.79	9.01 $\pm$ 1.08

with the best-performing competitor of this category (DER) achieving accuracy 3% lower on CIFAR-100 (83.94% vs. 86.95%) and almost 10% on ImageNet-R (66.73% vs. 76.36%) w.r.t. our two-stage variant, despite using a memory buffer of 5000 samples. Comparisons are striking for a reduced buffer of 1000 samples, with C-LN outperforming DER by almost 30% on ImageNet-R. We remark that our method does not store any old task data, being more viable in applications with privacy or memory constraints.

Comparing our approach with other rehearsal-free methods, C-LN still achieves the best accuracy, with a 2% improvement over CODA-Prompt and almost 6% over DualPrompt and L2P on the challenging ImageNet-R. On CIFAR-100, the gap between the models is reduced, with two-stage C-LN achieving the same accuracy as CODA-Prompt while still outperforming L2P (+3.3%) and DualPrompt (+5.6%). These results are remarkable, as the approaches share the same principles (e.g., selecting task-specific parameters) but they differ in the parameters tuned (prompts vs LayerNorm). Our results demonstrate the benefit of simple LayerNorm tuning vs. more complex strategies. It is also worth highlighting that C-LN single-stage and two-stage achieve comparable results, demonstrating that we can tradeoff the number of parameters for faster inference time without losing on the final performance.

While our model achieves good results w.r.t. accuracy, it shows higher forgetting than the prompt-learning strate-

gies, which might be a consequence of the stricter parameter isolation provided by the latter. Nevertheless, the higher accuracy shows that our model achieves a better stability/plasticity tradeoff, thus equally balancing the preservation of old knowledge and the learning of new concepts.

5-20 tasks on ImageNet-R. To further test both variants of our method, we performed experiments on ImageNet-R varying the number of incremental learning steps, reducing them to 5 (40 classes per task) and increasing them to 20 (10 classes per task), following [43]. The results are shown in Table 2 in terms of accuracy, for both our model and the prompt-based baselines. Even in these settings, our model achieves the best results, with the two-stage variant outperforming the previous state-of-the-art (CODA-Prompt) by almost 4% with 5 steps and 1% with 20. The same applies to our single-stage variant, outperforming CODA-Prompt by more than 3% with 5 tasks and almost 1% with 20. The gap is even more evident with L2P and DualPrompt, achieving results that are 7% (6% for the single-stage) lower than ours on 5 tasks, and 3% lower for the 20 tasks case. These results confirm that the effectiveness of our model and the benefits of learning task-specific LayerNorm generalizes across different scenarios, and it is not affected by the number of incremental learning steps or classes per step.

Computational cost. As analyzed in previous work [43], the number of learnable task-specific parameters plays a key

Table 2: Comparison on ImageNet-R by changing the number of tasks. The accuracy of our method is higher on average than the baseline methods.

Method	Number of tasks		
	5 $\uparrow$	10 $\uparrow$	20 $\uparrow$
UB	79.13	79.13	79.13
LN-Tune	81.18	81.18	81.18
L2P	72.12 $\pm$ 0.44	70.75 $\pm$ 0.23	67.12 $\pm$ 0.36
DualPrompt	71.83 $\pm$ 0.40	70.42 $\pm$ 0.21	68.15 $\pm$ 0.43
CODA-P	75.25 $\pm$ 0.15	74.00 $\pm$ 0.32	70.74 $\pm$ 0.41
C-LN Two-S	79.21$\pm$0.84	76.36$\pm$0.51	71.72$\pm$0.47
C-LN Single-S	78.43$\pm$0.67	76.25$\pm$0.79	71.62$\pm$0.38

role in the performance of rehearsal-free methods. Here we investigate whether the effectiveness of our model is linked to this aspect, reporting accuracy vs the number of trainable parameters in Table 3, for the ImageNet-R 10 tasks setting. As the table shows, our best-performing variant, i.e., the two-stage variant has a much lower number of learnable parameters than the previous state-of-the-art, CODA-Prompt (0.39M vs. 3.69M), despite achieving either comparable or superior results. While L2P uses much fewer parameters than our method (i.e., 0.05M), its performance has a large gap with ours (i.e., -5.6% accuracy). Notably, increasing the number of parameters alone does not ensure an increase in performance, with DualPrompt using more parameters than our approach but achieving results comparable to L2P. These results confirm the benefit of using LayerNorm in rehearsal-free continual learning, achieving the best trade-off between performance and number of parameters.

While all approaches discussed so far perform predictions via a two-stage pipeline, our single-stage variant provides prediction in a single forward pass. This results in a speed-up during inference, with our single-stage variant being $2\times$ faster than our two-stage one. This speed is reflected also during training, where we witnessed a $1.6\times$ speed-up compared to our two-stage variant. However, this advantage comes with a cost in terms of memory, almost doubling its requirement. Despite this, the single-stage variant still requires fewer parameters than DualPrompt and CODA-Prompt, while achieving either superior or comparable accuracy. Practically, one may pick one of the two C-LayerNorm variants based on the memory and/or speed required in the particular application scenario.

As a final note, it is worth highlighting that for the 5-task setting in Table 2, the performance of the Single-stage variant is lower than the two-stage one (i.e., 78.43% accuracy vs. 79.21%). This is surprising, as the single-stage variant has more flexibility in parameter selection since it might pick a different task ID at each layer. At the same time, we conjecture that this flexibility may cause an in-

Table 3: Accuracy vs. number of parameters in ImageNet-R 10-task benchmark. Our approach obtains a new state-of-the-art while reducing the number of parameters trained.

Method	Accuracy $\uparrow$	Param trained $\downarrow$
L2P	70.75 $\pm$ 0.23	0.05M
DualPrompt	70.42 $\pm$ 0.21	0.94M
CODA-P	74.00 $\pm$ 0.32	3.69M
C-LN Two-S	76.36$\pm$0.51	0.39M
C-LN Single-S	76.25$\pm$0.79	0.77M

herent distribution shift when multiple tasks are selected in subsequent layers, something that the classifier refinement step is not able to fix. While it is not trivial (i.e., as different layers focus on different features) encouraging a consistent task ID selection may further improve the performance of the single-stage variant.

4.3. Analyses of C-LayerNorm

Impact of the classifier refinement step. As described in Sec. 3.5, one of the key elements of our approach is the classifier refinement step. In this step, we train only the classifier while selecting the C-LN parameters with the same procedure adopted during inference. The idea of this step is to make the classifier more robust to potential mistakes in the selection of the parameters during inference. We analyze the impact of this choice in Figure 4, for the two-stage variant, showing the results on the 10-task settings without refinement (blue), with refinement (orange), and an oracle using the ground-truth task-specific parameters (green).

Comparing the performance of our model without refinement and the oracle, we can see how correct parameter selection matters. Despite the two models differing only on the C-LayerNorm’s parameter, the accuracy of the oracle is 4% higher on CIFAR-100 (90.16% vs. 86%) and 5% higher on ImageNet-R (79.51% vs. 74.4%) than the accuracy of our model without classifier refinement. These gaps suggest how mistakes in the parameters selection have a major impact on the final results, and should be taken into account in the model design.

In this context, our classifier refinement step partly addresses this issue, with an improvement of almost 2% on ImageNet-R and of 1% on CIFAR-100 over the base model not adopting it. These results are promising, as they show that, by exposing the classifier during training to potential selection errors, it becomes more robust. At the same time, there is still a large gap between our full model and the oracle, making this a promising direction for future work.

Learned task-specific parameters. One assumption behind using task-specific parameters for continual learning, is that each set of parameters captures the peculiarities of

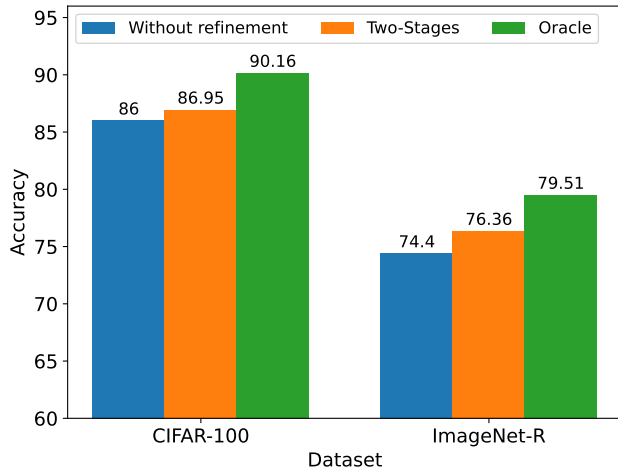


Figure 4: Classifier refinement effect. By training the classifier using inferred task identities instead of the ground truth ones, we improve the performance for both CIFAR-100 and ImageNet-R.

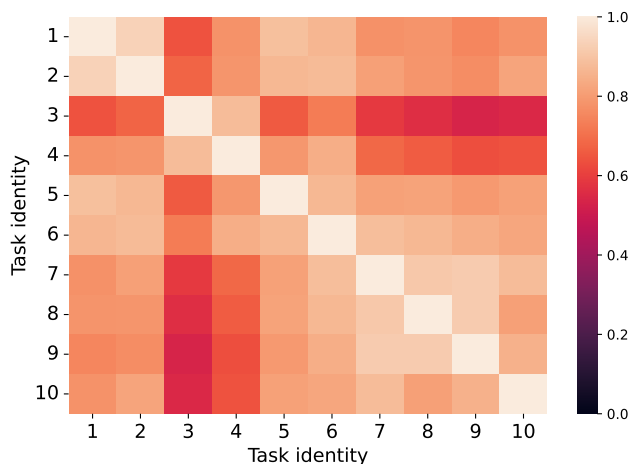


Figure 5: Similarity matrix of selection parameters on ImageNet-R 10-task benchmark. The learned selection vectors have a high inter-task similarity. That results in high pairwise similarity scores between the learned vectors.

the dataset it is exposed to. Here, we test this assumption, providing qualitative analyses of the parameters' similarity at different layers. The results are shown in Figure 6, where we report a t-SNE [44] of the scale and bias parameters of LayerNorm at different ViT layers (initial, middle, end, i.e., 1, 12, 25), for our two-stage variant and the 10-task ImageNet-R benchmark. As the figure shows, there is a pattern in the similarity of the parameters across tasks, with neighboring tasks (i.e., learned sequentially) being generally close in terms of learned parameters. This applies to both the scale and bias of LayerNorm. This behavior hints

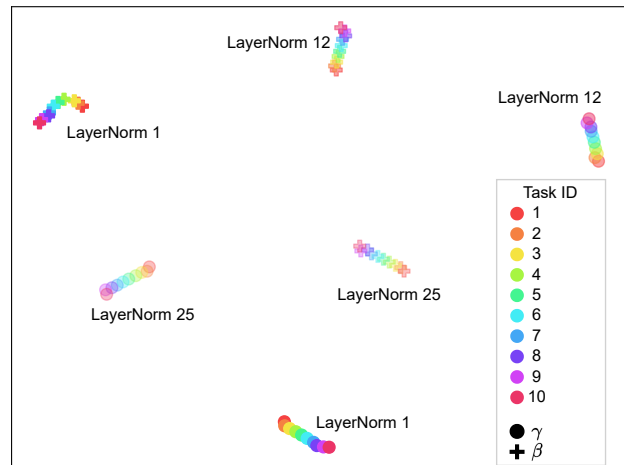


Figure 6: t-SNE on the learned weights and biases in the 1st, 12th, and 25th LayerNorm layers in our approach. Representations have a high inter-task similarity. Such a property is desirable since the Task Identifier introduces errors in the weight selection. Such similar representations attenuate the error's magnitude, thus achieving consistent performances.

that these tasks need similar feature extractors and that subsequent tasks are more similar than randomly picked pairs.

We verify this in Figure 5, where we report the similarity between the task keys, used to select the parameters during inference. The heatmap shows how the similarity is indeed higher for tasks presented in order (e.g., task 1 with task 2, task 3 with task 4, etc.). The fact that the similarity between tasks/keys is reflected in the parameters space (Figure 6), suggests that future work may use directly the parameters and the network's activations to estimate the task ID during inference (e.g., by comparing normalization statistics).

5. Conclusion

In this paper, we present Continual LayerNorm, a new method for rehearsal-free continual learning. Contrary to prompt-based approaches, we propose a simple strategy and tune the LayerNorms parameters of the pre-trained Vision Transformer. Moreover, we present a single-stage variant to halve the inference time and a training strategy to improve the classifier robustness during inference. Experiments on CIFAR-100 and ImageNet-R show that our method either outperforms or is on par with the current state-of-the-art while being computationally cheaper.

Acknowledgements. This work was supported by the MUR PNRR project FAIR - Future AI Research (PE00000013) funded by the NextGenerationEU, the PRIN project LEGO-AI (Prot. 2020TA3K9N), and the EU H2020 AI4Media No. 951911 project. It was carried out in the Vision and Learning joint laboratory of FBK and UNITN.

References

- [1] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *ECCV*, 2018. 2
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 2, 3
- [3] Hyojin Bahng, Ali Jahanian, Swami Sankaranarayanan, and Phillip Isola. Exploring visual prompts for adapting large-scale models. *arXiv preprint arXiv:2203.17274*, 2022. 2
- [4] Samyadeep Basu, Daniela Massiceti, Shell Xu Hu, and Soheil Feizi. Strong baselines for parameter efficient few-shot fine-tuning. *arXiv preprint arXiv:2304.01917*, 2023. 2, 3, 5
- [5] Rodrigo Berriel, Stephane Lathuillere, Moin Nabi, Tassilo Klein, Thiago Oliveira-Santos, Nicu Sebe, and Elisa Ricci. Budget-aware adapters for multi-domain learning. In *ICCV*, 2019. 2
- [6] Hakan Bilen and Andrea Vedaldi. Universal representations: The missing link between faces, text, planktons, and cat breeds. *arXiv preprint arXiv:1701.07275*, 2017. 2, 3
- [7] Adrian Bulat, Jean Kossaifi, Georgios Tzimiropoulos, and Maja Pantic. Incremental multi-domain learning with network latent tensor factorization. In *AAAI*, 2020. 2
- [8] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. In *NeurIPS*, 2020. 5
- [9] Francisco M Castro, Manuel J Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. End-to-end incremental learning. In *ECCV*, 2018. 2
- [10] Fabio Cermelli, Massimiliano Mancini, Samuel Rota Buló, Elisa Ricci, and Barbara Caputo. Modeling the background for incremental learning in semantic segmentation. In *CVPR*, 2020. 2
- [11] Fabio Cermelli, Massimiliano Mancini, Samuel Rota Buló, Elisa Ricci, and Barbara Caputo. Modeling the background for incremental and weakly-supervised semantic segmentation. *IEEE T-PAMI*, 44(12):10099–10113, 2021. 2
- [12] Hyuntak Cha, Jaeho Lee, and Jinwoo Shin. Co2l: Contrastive continual learning. In *ICCV*, 2021. 2, 5
- [13] Arslan Chaudhry, Puneet K Dokania, Thalaiyasingam Ajanthan, and Philip HS Torr. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *ECCV*, 2018. 2, 5
- [14] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc Aurelio Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019. 5
- [15] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 5
- [16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 1, 5
- [17] Enrico Fini, Victor G Turrissi Da Costa, Xavier Alameda-Pineda, Elisa Ricci, Karteek Alahari, and Julien Mairal. Self-supervised models are continual learners. In *CVPR*, 2022. 2
- [18] Enrico Fini, Stéphane Lathuillere, Enver Sangineto, Moin Nabi, and Elisa Ricci. Online continual learning under extreme memory constraints. In *ECCV*, 2020. 1, 2
- [19] Yasir Ghunaim, Adel Bibi, Kumail Alhamoud, Motasem Alfarra, Hasan Abed Al Kader Hammoud, Ameya Prabhu, Philip HS Torr, and Bernard Ghanem. Real-time evaluation in online continual learning: A new hope. In *CVPR*, 2023. 1, 2
- [20] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, et al. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *ICCV*, 2021. 2, 5
- [21] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *ICML*, 2019. 2
- [22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022. 2
- [23] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *ECCV*, 2022. 2
- [24] Ronald Kemker, Marc McClure, Angelina Abitino, Tyler Hayes, and Christopher Kanan. Measuring catastrophic forgetting in neural networks. In *AAAI*, 2018. 1
- [25] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015. 5
- [26] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the national academy of sciences*, 2017. 2
- [27] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 2, 5
- [28] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, 2021. 5
- [29] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL*, 2021. 2, 5
- [30] Yunsheng Li and Nuno Vasconcelos. Efficient multi-domain learning by covariance normalization. In *CVPR*, 2019. 2
- [31] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE T-PAMI*, 40(12):2935–2947, 2017. 2
- [32] Yaoyao Liu, Yuting Su, An-An Liu, Bernt Schiele, and Qianru Sun. Mnemonics training: Multi-class incremental learning without forgetting. In *CVPR*, 2020. 1, 2

- [33] Zilin Luo, Yaoyao Liu, Bernt Schiele, and Qianru Sun. Class-incremental exemplar compression for class-incremental learning. In *CVPR*, 2023. 1, 2
- [34] Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. 2
- [35] Massimiliano Mancini, Elisa Ricci, Barbara Caputo, and Samuel Rota Bulò. Adding new tasks to a single network with weight transformations using binary masks. In *ECCV-WS*, 2018. 2
- [36] Massimiliano Mancini, Elisa Ricci, Barbara Caputo, and Samuel Rota Bulò. Boosting binary masks for multi-domain learning through affine transformations. *MVA*, 31(6):42, 2020. 2
- [37] Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*. 1989. 1
- [38] Ameya Prabhu, Hasan Abed Al Kader Hammoud, Puneet K Dokania, Philip HS Torr, Ser-Nam Lim, Bernard Ghanem, and Adel Bibi. Computationally budgeted continual learning: What does matter? In *CVPR*, 2023. 2
- [39] Ameya Prabhu, Philip HS Torr, and Puneet K Dokania. Gdumb: A simple approach that questions our progress in continual learning. In *ECCV*, 2020. 1, 2, 5
- [40] Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *NeurIPS*, 2017. 2
- [41] Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Efficient parametrization of multi-domain deep neural networks. In *CVPR*, 2018. 2
- [42] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *CVPR*, 2017. 1, 2
- [43] James Seale Smith, Leonid Karlinsky, Vyshnavi Gutta, Paola Cascante-Bonilla, Donghyun Kim, Assaf Arbelle, Rameswar Panda, Rogerio Feris, and Zsolt Kira. Coda-prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning. In *CVPR*, 2023. 1, 2, 3, 4, 5, 6
- [44] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *JMLR*, 9(11), 2008. 8
- [45] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, et al. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *ECCV*, 2022. 1, 2, 5
- [46] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning. In *CVPR*, 2022. 1, 2, 3, 5
- [47] Yue Wu, Yinpeng Chen, Lijuan Wang, Yuancheng Ye, Zicheng Liu, Yandong Guo, and Yun Fu. Large scale incremental learning. In *CVPR*, 2019. 5
- [48] Lu Yu, Bartłomiej Twardowski, Xialei Liu, Luis Herranz, Kai Wang, Yongmei Cheng, Shangling Jui, and Joost van de Weijer. Semantic drift compensation for class-incremental learning. In *CVPR*, 2020. 2
- [49] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017. 2
- [50] Yaqian Zhang, Bernhard Pfahringer, Eibe Frank, Albert Bifet, Nick Jin Sean Lim, and Yunzhe Jia. A simple but strong baseline for online continual learning: Repeated augmented rehearsal. In *NeurIPS*, 2022. 2
- [51] Fei Zhu, Xu-Yao Zhang, Chuang Wang, Fei Yin, and Cheng-Lin Liu. Prototype augmentation and self-supervision for incremental learning. In *CVPR*, 2021. 1, 2